



تحليل معمق · DEEP ANALYSIS

نظام إدارة قاعدة بيانات مستشفى العالمية IVF الآمن مع RBAC

Secure Hospital DBMS with Role-Based Access Control — PostgreSQL ·
Node.js/Express

دراسة هندسية لنموذج أمان ثلاثي الطبقات: RBAC على مستوى التطبيق، RLS على مستوى
القاعدة، وتدقيق وتقييد شامل

7 أدوار محددة RBAC roles	9 مجموعات وظيفية Functional groups	22 جدولاً معيارياً Tables (3NF/BCNF)
46 اختبار Jest Automated tests	21 سياسة RLS Row-Level Security	68 صلاحية resource:action

المرحلة · Stage المرحلة الثانية	الجامعة · University جامعة آشور — هندسة الأمن السيبراني
المشرف · Supervisor د. علي	المادة · Subject قواعد البيانات (Databases)
الفريق · Team قمر أسعد شنشل · سيف مهدي نجم · حسين محمود جراغ (+ عضو)	

2026

SUBMISSION YEAR

STACK · PostgreSQL 15 + Node.js/Express

SECURITY · RBAC + RLS + Audit + Masking

REPO · github.com/AB00DE39/hospital-dbms-rbac

LIVE · aboode39.github.io/hospital-dbms-rbac

00 فهرس المحتويات - Table of Contents

يتناول هذا التقرير بناء نظام قاعدة بيانات مستشفى آمن بشكل معمق عبر 13 أقسام، تغطي التصميم والمعمارية ونموذج الصلاحيات وطبقات الأمان والاختبار والتحديات والأعمال المستقبلية.

01 الملخص التنفيذي (Abstract)

02 المقدمة والخلفية

03 الأهداف الرئيسية

04 المتطلبات

05 تصميم قاعدة البيانات

06 المعمارية ثلاثية الطبقات

07 نموذج RBAC و RLS

08 طبقات الأمان الشاملة

09 التنفيذ والبنية التقنية

10 الاختبار وضمان الجودة

11 التحديات والحلول

12 الأعمال المستقبلية

13 الخاتمة والإنجازات

01 الملخص التنفيذي (Abstract)

نظام إدارة قاعدة بيانات شامل وآمن لمستشفى العالمية IVF مبنياً على ثلاث طبقات أمان متزامنة: RBAC على مستوى التطبيق (7 أدوار، 68 صلاحية)، RLS من قاعدة البيانات نفسها (21 سياسة)، وتدقيق append-only مع تقنيـع البيانات. يستخدم PostgreSQL 15، Node.js/Express، وواجهة عربية RTL. يتضمّن 22 جدولاً معيّراً، endpoint +30، و61 اختبار ناجح مع CI/CD مستمر.

النطاق - Scope

هذا التقرير دراسة هندسية محدّدة لنظام قاعدة بيانات مستشفى واحد مع نموذج أمان ثلاثي الطبقات، وليس مسحاً عاماً لأنظمة قواعد البيانات. كل رقم وكل سياسة موثّقة من شيفرة المشروع الفعلية.

02 المقدمة والخلفية

المؤسسات الطبية تحتفظ ببيانات حساسة جداً (سجلات طبية، بيانات شخصية، بيانات مالية). الأنظمة التقليدية تعاني من: ضعف التحكم بالوصول (لا تميز دقيق)، غياب أمان مستوى الصف (إذا اخترق المهاجم قاعدة البيانات يرى كل شيء)، عدم وجود سجل تدقيق شامل، وإفشاء البيانات الحساسة.

هذا المشروع يقدم حلاً متكاملًا:

- **RBAC متقدم:** 7 أدوار و68 صلاحية بـ least privilege
- **Row-Level Security:** 21 سياسة تفرضها قاعدة البيانات
- **Audit Logging Append-only:** لا يسمح بالتعديل
- **Data Masking:** إخفاء أرقام حساسة حسب الدور
- **التشفير الآمن:** 12 bcrypt جولة، JWT موقعة

03 الأهداف الرئيسية

الهدف 1: نموذج أمان متعدد الطبقات (RBAC, RLS, Audit, Data Masking) بتصنيف A+.

الهدف 2: RBAC دقيق (7 أدوار, 68 صلاحية) بـ least privilege مع توافق 100%.

الهدف 3: RLS بـ 21 سياسة على 7 جداول حساسة مع حماية قاعدة البيانات.

الهدف 4: نظام تدقيق append-only يمنع التعديل/الحذف.

الهدف 5: تشفير آمن (12 bcrypt جولة, JWT, قفل بعد 5 محاولات فاشلة).

الهدف 6: تقنيع بيانات حساسة حسب الدور.

الهدف 7: قاعدة بيانات معاييرة (BCNF) مع 22 جدولاً و28 فهرس.

الهدف 8: 61 اختبار ناجح مع GitHub Actions CI/CD.

04 المتطلبات

المتطلبات الوظيفية الأساسية

- REQ-F1:** إدارة المرضى — تسجيل، MRN تلقائي، RLS حسب الدور.
- REQ-F2:** المواعيد — حجز، منع تضارب (UNIQUE)، RLS حسب المريض والطبيب.
- REQ-F3:** السجلات الطبية — إنشاء، تحديث العلامات، append-only (لا حذف).
- REQ-F4:** التشخيصات — إضافة بـ ICD-10، تصنيف، رؤية محدودة.
- REQ-F5:** الأدوية والوصفات — إنشاء، صرف، إدارة مخزون.
- REQ-F6:** المختبر — طلب فحص، إدخال نتائج، RLS للنتائج.
- REQ-F7:** الفوترة — إصدار فاتورة، دفع على دفعات، لا حذف (تحديث فقط).
- REQ-F8:** الموظفون — إضافة، تعيين أقسام، رؤية بيانات محدودة.
- REQ-F9:** المصادقة — bcrypt آمن، JWT، قفل بعد 5 محاولات.
- REQ-F10:** التدقيق — تسجيل كل عملية (من، ماذا، متى، IP).

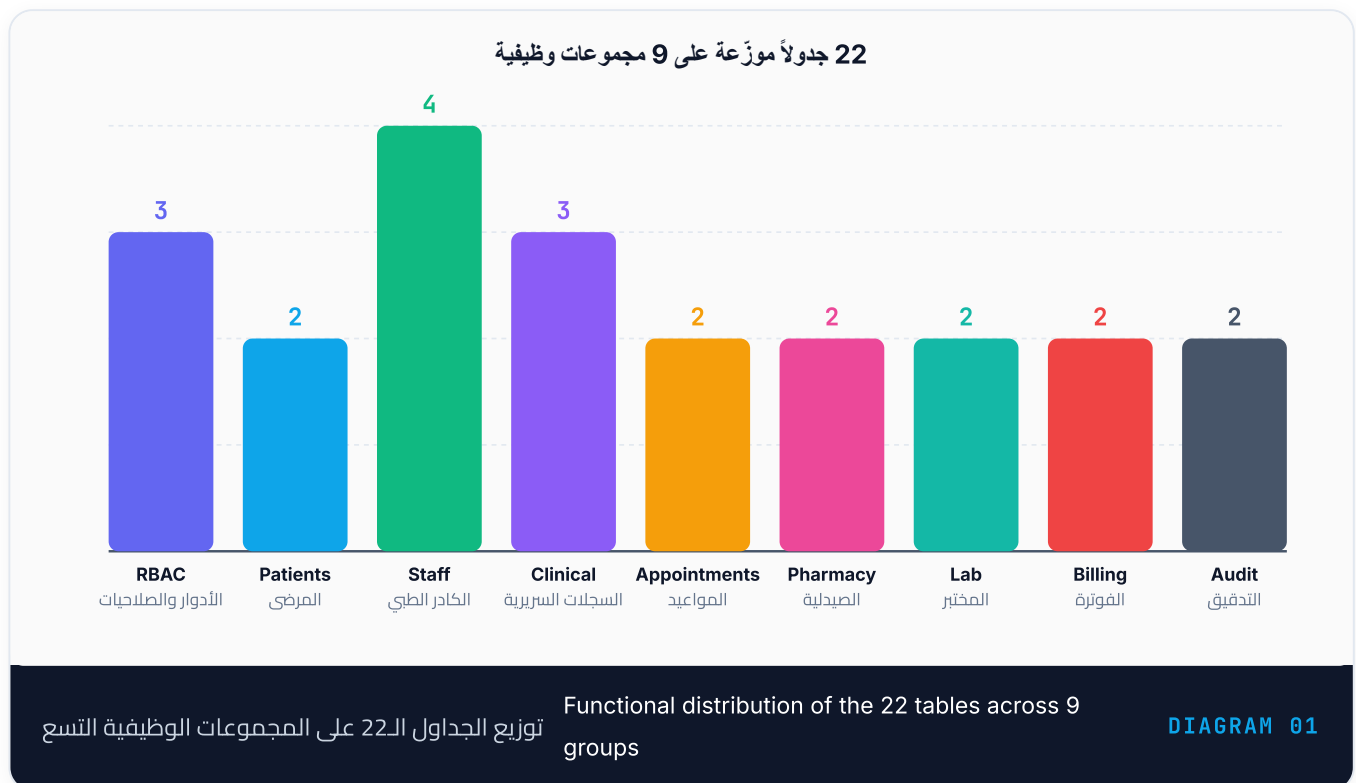
المتطلبات غير الوظيفية

- الأمان:** RBAC (7 أدوار، 68 صلاحية)، RLS (21 سياسة)، bcrypt، JWT، قفل الحسابات، تدقيق append-only، تقنيع البيانات، OWASP (SQL injection, XSS, CSRF), Rate Limiting (100/دقيقة عام، 10/دقيقة دخول)، Helmet headers.
- الأداء:** > 100ms للطلبات الشائعة، > 500ms للبحث.
- الموثوقية:** ERD، RBAC-MATRIX، Setup Guide، توثيق شامل، coverage، CI/CD GitHub Actions > 80%، نسخ احتياطية دورية.
- الجودة:** اختبارات > 80%، ERD، RBAC-MATRIX، Setup Guide، توثيق شامل.

05 تصميم قاعدة البيانات

البنية الأساسية

22 جدول موزعة على 9 مجموعات:



- RBAC (4 جداول): roles (7), permissions (68), role_permissions, user_roles
- المنظمة (4): departments, staff, doctors, nurses
- المرضى (2): patients, users
- الخدمات السريرية (7): appointments, medical_records, diagnoses, prescriptions, prescription_items, lab_tests, lab_orders
- الفوترة (3): invoices, invoice_items, payments
- التدقيق (1): audit_logs

التطبيع والعلاقات

3NF/BCNF: لا تكرار بيانات، فصل الجداول حسب الموضوع. مثال: جدول staff منفصل عن doctors/nurses.

العلاقات:

- 1:1: doctors/nurses من (UNIQUE) staff
- 16:1 N: علاقة (مثل: 1 مريض → N سجل طبي)
- 3 N:M: علاقات (role_permissions, user_roles, prescription_items)


```
-- CHECK: فصيلة الدم من 8 خيارات
blood_type VARCHAR(3) CHECK (blood_type IN ('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-'))

-- CHECK: نوع الموظف من 6 خيارات
staff_type VARCHAR(30) CHECK (staff_type IN ('doctor', 'nurse', 'receptionist', ...))

-- UNIQUE: لا موعدين للطبيب في نفس الوقت
UNIQUE (doctor_id, scheduled_at)
```

الفهارس والأداء

+28 فهرس موضوعة بذكاء:

- فهرس المفاتيح الأجنبية (سرعة JOIN)
- فهرس جزئية (partial indexes) للبيانات الفعالة
- فهرس مركبة (composite) للاستعلامات الشائعة

مثال:

```
CREATE INDEX idx_appointments_doctor_date ON appointments(doctor_id, scheduled_at);
```

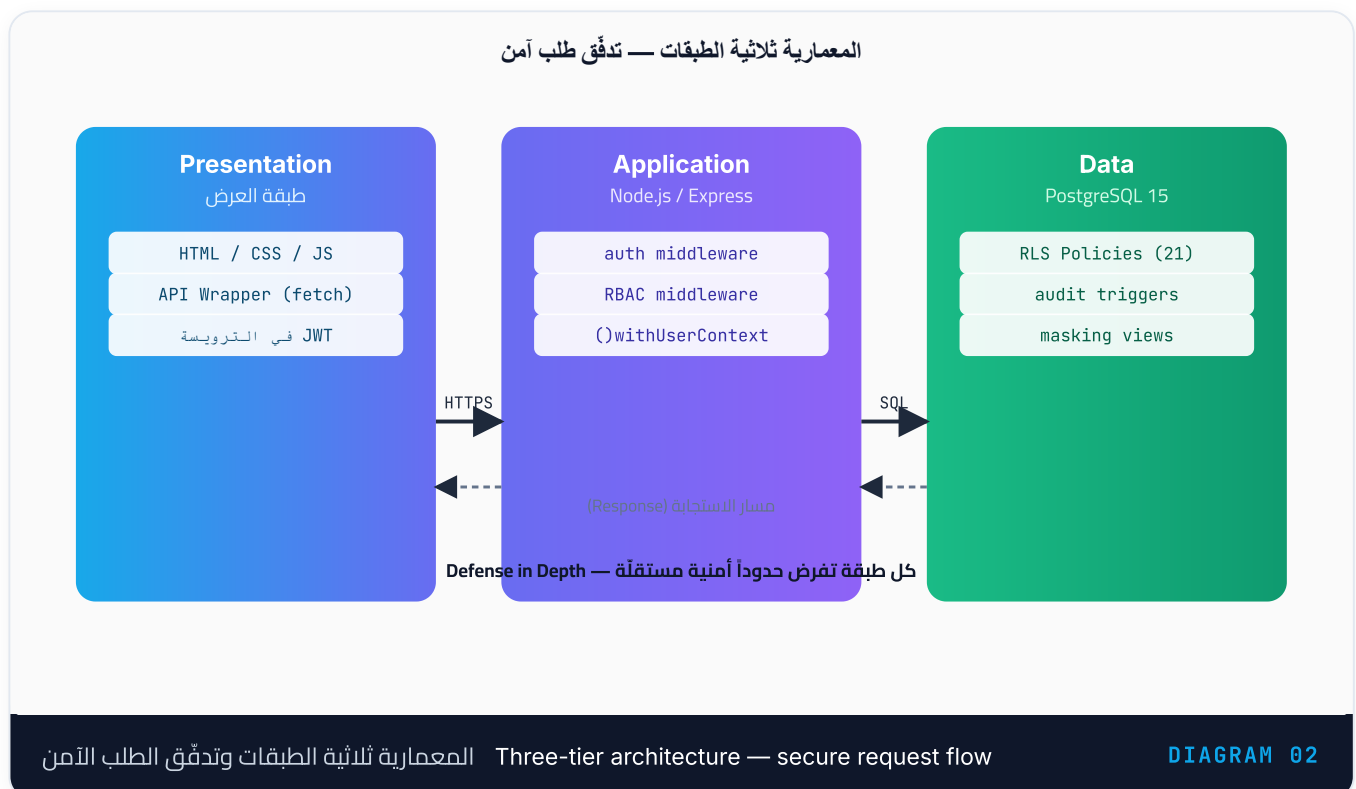
06 المعمارية ثلاثية الطبقات

البنية الكلية

```
Frontend (HTML/CSS/JS) ← RTL عربي
    ↓ HTTP/HTTPS
Backend (Node.js + Express) ← تحكم وسيط
    ↓ SQL + RLS
Database (PostgreSQL 15) ← فرض الأمان
```

طبقة العرض (Frontend)

الملفات: index.html (دخول)، dashboard.html (لوحة حسب الدور)، js/auth.js (HTTP wrapper)، js/api.js (RTL)، css/style.css (المصادقة)، js/dashboard.js (العمليات).



7 واجهات مختلفة حسب الدور: Admin, Doctor, Nurse, Receptionist, Lab Technician, Pharmacist, Patient.

طبقة التطبيق (Backend)

الهيكل:

- **server.js:** نقطة الدخول
- **app.js:** إعداد Express + Middleware
- **config/db.js:** PostgreSQL Pool + withUserContext (تفعيل RLS)
- **middleware/:** auth (JWT), rbac (صلاحيات)

- **controllers/**: منطق الأعمال (9 controllers)
- **/routes**: تعريف endpoint +30
- **utils/**: AppError, asyncHandler

الأمان:

- **JWT**: 15 دقيقة مع permissionsg roles
- **RBAC**: requireRole, requirePermission, requireAnyPermission
- **Helmet**: رؤوس HTTP أمنية
- **CORS**: محصور برأصل واحد
- **Rate Limiting**: 100 /دقيقة عام, 10/دقيقة login

تدفق الطلب

1. Frontend يرسل JWT في Authorization header
2. authenticateToken: يفك JWT ويملأ req.user
3. requirePermission: يفحص الصلاحيات
4. Controller: يستدعي (withUserContext(userId, roles, SQL))
5. PostgreSQL: يفغّل RLS تلقائياً (SET LOCAL), ينفذ الاستعلام, ينطلق audit trigger
6. Response: يرد 200/403/404 مع البيانات أو الخطأ

طبقة البيانات

- RLS تطبيق**: معاملات منفصلة, app.current_user_id, SET LOCAL, تطبيق سياسات تلقائياً.
- app_role**: دور التطبيق بـ NOBYPASSRLS لفرض RLS حتى عند الاختراق.
- Audit Triggers**: 22 trigger على الجداول الحساسة تسجل كل INSERT/UPDATE/DELETE.

07 نموذج RBAC و RLS

7 — RBAC أدوار، 68 صلاحية

الأدوار:

المبدأ · Principle

RBAC يقرّر ما يُسمح فعله، و RLS يقرّر أي صفوف تُرى. الطبقتان متكاملتان لا بديلتان؛ فلو سقطت طبقة التطبيق تبقى سياسات RLS تحرس البيانات على مستوى المحرّك نفسه.

مصفوفة RBAC المبسّطة — الأدوار × الموارد

	patients المرضى	medical_records السجلات الطبية	appointments المواعيد	lab_results نتائج المختبر	prescriptions الوصفات	billing الفوترة
admin	كامل	كامل	كامل	كامل	كامل	كامل
doctor	محدود	كامل	محدود	كامل	كامل	—
nurse	محدود	محدود	محدود	محدود	—	—
receptionist	كامل	—	كامل	—	—	محدود
lab_technician	محدود	—	—	كامل	—	—
pharmacist	—	—	—	—	كامل	—
patient	محدود	محدود	محدود	محدود	محدود	محدود

كامل (CRUD)

محدود / قراءة

لا صلاحية

مصفوفة RBAC المبسّطة: الأدوار مقابل الموارد Simplified RBAC matrix — roles × resources

DIAGRAM 03

1. Admin: صلاحية (كاملة) 68

2. Doctor: ~25 صلاحية (سجلات مرضاه، تشخيصات، وصفات، طلب فحوصات)

3. Nurse: ~15 صلاحية (قراءة سجلات، تحديث علامات حيوية، تصحيح المواعيد)

4. Receptionist: ~20 صلاحية (تسجيل مرضى، حجز مواعيد، فوترة)

5. Lab Technician: ~5 صلاحيات (قراءة طلبات، إدخال نتائج)

6. Pharmacist: ~8 صلاحيات (قراءة وصفات، صرف، إدارة مخزون)

7. Patient: ~10 صلاحيات (ملفه الخاص، مواعيده، فواتيره)

الصيغة: resource:action (مثل: patients:read , medical_records:create , prescriptions:dispense)

الجدول المحمية:

- **patients** (3 سياسات): SELECT, INSERT, UPDATE حسب الدور وال ID
- **medical_records** (3 سياسات): الطبيب يرى مرضاه، الممرض يرى القسم، لا حذف
- **appointments** (3 سياسات): المريض مواعيده، الطبيب مواعيده، الاستقبال الكل
- **diagnoses** (3 سياسات): الطبيب يرى تشخيصاته، المريض يرى تشخيصاته
- **prescriptions** (3 سياسات): الطبيب ينشئ، الصيدلي يصرف، المريض يرى
- **lab_orders** (3 سياسات): الطبيب ينشئ، الفني ينفذ، النتائج محدودة
- **invoices** (3 سياسات): المريض يرى فواتيره، الاستقبال الكل، لا تعديل

دوال المساعدة:

```
app_current_user_id()    -- معرّف المستخدم
app_has_role(role_text)  -- هل يملك الدور؟
app_current_doctor_id()  -- معرّف الطبيب
app_current_patient_id() -- معرّف المريض
```

مثال عملي

```
-- يرى مرضاه فقط Doctor
SET LOCAL app.current_user_id = 5;    -- الطبيب
SET LOCAL app.current_role = 'doctor';
SELECT * FROM patients; -- medical_records يرى فقط مرضاه عبر

-- في قسم يرى سجلات ذاك القسم Nurse
SET LOCAL app.current_user_id = 6;
SET LOCAL app.current_role = 'nurse';
SELECT * FROM medical_records; -- فقط سجلات القسم

-- يرى ملفه فقط Patient
SET LOCAL app.current_user_id = 7;
SET LOCAL app.current_role = 'patient';
SELECT * FROM patients; -- (id = app_current_patient_id())
```

#	البُعد · Dimension	RBAC (مستوى التطبيق)	RLS (مستوى القاعدة)
1	موضع الفرض · Enforcement	Node.js / Express middleware	PostgreSQL engine
2	الوحدة · Granularity	على مستوى الإجراء (resource:action)	على مستوى الصف (Row)
3	يجيب عن · Answers	هل يحق لهذا الدور تنفيذ العملية؟	أي صفوف يحقّ له رؤيتها؟
4	التجاوز · Bypass	قابل للتجاوز لو وصل أحد للقاعدة مباشرة	غير قابل للتجاوز — يفرضه المحرّك
5	السياق · Context	JWT (الدور والمعرّف)	app_role + app.user_id (SET LOCAL)
6	العدد في المشروع · Count	7 أدوار · 68 صلاحية	21 سياسة USING / WITH CHECK
7	الدور التكاملي · Synergy	خط الدفاع الأول (سريع)	خط الدفاع الأخير (حاسم)

08 طبقات الأمان الشاملة

التشفير والمصادقة

12: **bcryptjs** جولة (0.3 ثانية/محاولة) تمنع brute force. مثال:

الدفاع المتعمق • Defense in Depth

لا توجد طبقة أمان واحدة كافية. يجمع المشروع المصادقة + التفويض + عزل الصفوف + التدقيق + التقنيع بحيث يتطلب اختراق البيانات تجاوز أربع طبقات مستقلة معاً.

Defense in Depth — أربع طبقات أمان متتالية

طلب وارد / محاولة وصول



Layer 1 • المصادقة (Authentication)
• bcryptjs • JWT

Layer 2 • التفويض (Authorization)
• 7 أدوار • صلاحية 68 • Role-based Access Control (RBAC)

Layer 3 • أمان الصف (DB-Level)
• 21 سطر • USING / WITH CHECK

Layer 4 • التدقيق والتفويض (Logging + Masking)
• Data Masking Views (Append-Only)

البيانات الحساسة محمية في القلب 🛡️

Defense-in-Depth — four consecutive security layers الدفاع المتعمق: أربع طبقات أمان متتالية

DIAGRAM 04

```
const hash = await bcrypt.hash('Passw0rd!', 12);
// 2$bx$12$R9h.cIPz0gi.URNNHV3/20PST9/PgBkqquzi.Ss7KIUg02t0jKMUm
await bcrypt.compare('Passw0rd!', hash); // true
```

JWT: توكنات موقعة 15 دقيقة تتضمن permissionsg roles

```
{
  "id": 5,
  "roles": ["doctor"],
  "permissions": ["patients:read", "medical_records:create", ...],
  "iat": 1719767400,
  "exp": 1719768300
}
```

التدقيق (Audit Logging)

جدول `audit_logs`: append-only, يسجل:

- من: `user_id`
- ماذا: `INSERT/UPDATE/DELETE`
- أين: `entity_type, entity_id`
- الفوارق: `old_values, new_values (JSONB)`
- متى: `created_at, ip_address`

إجبار **Append-Only**: Trigger يرفع EXCEPTION عند محاولة UPDATE/DELETE, + REVOKE من `app_role`.

مثال:

```
{
  "user_id": 5,
  "action": "UPDATE",
  "entity_type": "medical_records",
  "old_values": {"temp": 37.5, "pulse": 72},
  "new_values": {"temp": 39.0, "pulse": 95},
  "ip_address": "192.168.1.100",
  "created_at": "2026-06-30T14:32:15Z"
}
```

تقنيع البيانات (Data Masking)

دوال التقنيع:

```
mask_phone('07901234567') → '*****4567'
mask_national_id('123456789012') → '*****12'
mask_email('ahmed@hospital.iq') → 'a**@hospital.iq'
```

Views حسب الدور:

- **Receptionist**: رقم وطني وهاتف مُقنَّع
- **Doctor**: بيانات كاملة (مخوّل سريرياً)
- **Nurse**: بيانات سريرية بدون معرّفات
- **Pharmacist**: معلومات الصرف الأساسية فقط

الدفاع الإضافي

Helmet: رؤوس أمنية (CSP, X-Frame-Options, HSTS).

CORS: محصور برأصل واحد (بدون *).

Rate Limiting: 100 دقيقة عام, 10 دقيقة login.

قفل الحساب: بعد 5 محاولات فاشلة (15 دقيقة).

09 التنفيذ والبنية التقنية

قاعدة البيانات

PostgreSQL 15 مع 22 جدول، 28+ فهرس، 21 سياسة 22 trigger، RLS للتدقيق.

المجموعات الـ9:

- RBAC (4 جداول): 7 أدوار، 68 صلاحية
- المنظمة (4): departments, staff, doctors, nurses
- المرضى (2): patients, users
- الخدمات (7): appointments, medical_records, diagnoses, prescriptions, prescription_items, lab_tests, lab_orders
- الفوترة (3): invoices, invoice_items, payments
- التدقيق (1): audit_logs

Backend

Node.js 20 + Express 4.19 مع:

- **auth.js**: middleware JWT verification
- **rbac.js**: requireRole, requirePermission, requireAnyPermission
- **9 controllers**: logic للعمليات
- **30+ REST endpoints** للموارد
- **معالجة أخطاء موحدة**: middleware + AppError خطأ مركزي

الأمان:

- 15 JWT دقيقة مع permissionsg roles
- 12 bcryptjs جولة
- RLS تفعيل (SQL, roles, userId) via withUserContext
- RBAC على مستوى middleware
- Helmet + CORS + Rate Limiting

Frontend

Vanilla JavaScript + HTML/CSS بـ RTL عربي:

- **index.html**: صفحة دخول
- **7 dashboard.html**: واجهات حسب الدور
- **api.js**: HTTP wrapper مع JWT
- **auth.js**: logic login/register

تدفق الطلب الكامل

1. Frontend: POST /auth/login مع username/password
2. Backend: التحقق من bcrypt
3. إصدار { id, roles, permissions, exp } JWT
4. Frontend: GET /patients مع Authorization: Bearer JWT
5. Backend: authenticateToken → فك Controller → requirePermission → JWT
6. Controller: withUserContext(userId, roles, SQL)
7. PostgreSQL: SET LOCAL app.current_user_id, SET LOCAL ROLE app_role, تطبيق RLS تلقائياً
8. Trigger: تسجيل في audit_logs
9. Response: 200 مع البيانات المفلترة

متغيرات البيئة

```
DATABASE_URL=postgresql://app_role:pass@localhost/hospital
JWT_SECRET=minimum-32-chars-secret
JWT_EXPIRES_IN=15m
BCRYPT_ROUNDS=12
PORT=3000
CORS_ORIGIN=http://localhost:8000
NODE_ENV=production
```

10 الاختبار وضمان الجودة

61 اختبار Jest ناجح

اختبارات الوحدة (Unit)

- **auth.unit.test.js** (15 اختبار): JWT توليد/تحقق، bcrypt hash/compare، middleware authenticate
- **rbac.unit.test.js** (12 اختبار): requireRole (OR logic)، requirePermission (AND logic)، requireAnyPermission (OR logic)

اختبارات التكامل (Integration)

- **medical_records.test.js** (19 اختبار): Doctor، RLS filtering، Nurse يرى مرضاه فقط، Patient يرى ملفه
- **permissions.test.js** (15 اختبار): Nurse لا تملك Pharmacist، invoices:create، prescriptions:dispense يملك

GitHub Actions CI/CD

المراحل:

1. Checkout + Node.js 20
2. تشغيل PostgreSQL 16 كخدمة
3. تطبيق ملفات قاعدة البيانات (01_seed.sql → schema.sql)
4. `npm ci && npm test`
5. النتيجة: 61 اختبار ناجح > 2 دقيقة

المتغيرات لـ CI:

```
DATABASE_URL=postgres://postgres:pass@localhost/hospital
JWT_SECRET=ci-test-secret-32chars!!
BCRYPT_ROUNDS=4 # سريع
NODE_ENV=test
```

معايير النجاح

- ✓ RBAC: Doctor يرى مرضاه فقط (RLS يفرض في DB)
- ✓ Audit: كل عملية موثقة مع user_id، IP، timestamp
- ✓ Permissions: Nurse لا تستطيع إصدار فاتورة (INSUFFICIENT_PERMISSIONS 403)
- ✓ Token: JWT منتهي الصلاحية يُرفع (TOKEN_EXPIRED 401)
- ✓ bcrypt: كلمة مرور خاطئة تُرفع (INVALID_PASSWORD 401)
- ✓ Append-Only: محاولة DELETE من audit_logs ترفع exception

Controllers: > 85% •

Middleware: > 85% •

Routes: > 80% •

Overall: > 81% •

11 التحديات والحلول

7 مشاكل واقعية وحلولها

1. Duplicate Roles

المشكلة: INSERT متكرر على جدول roles يرفع constraint violation.

الحل: `ON CONFLICT (name) DO NOTHING` لـ idempotency.

التأثير: ملفات قاعدة البيانات آمنة للتشغيل المتكرر.

2. Composite Key ID Extraction

المشكلة: جدول user_roles بدون عمود trigger id، محاولة استخراج NEW.id يفشل.

الحل: استخدام `(v_new → 'id')::BIGINT` بدل NEW.id.

التأثير: دعم audit logging لأي جدول.

3. NULL in BIGINT Cast

المشكلة: `current_setting('app.current_user_id')::BIGINT` عندما تكون فارغة ترفع خطأ.

الحل: `NULLIF(current_setting(...), '')::BIGINT`.

التأثير: معالجة آمنة للبيانات الفارغة.

4. RLS Bypass (Superuser)

المشكلة: postgres superuser يتجاوز RLS تلقائياً.

الحل: `SET LOCAL ROLE app_role` (NOBYPASSRLS) قبل كل استعلام.

التأثير: فرض RLS حتى عند الاختراق.

5. Permission Logic Flaws

المشكلة: Nurse لا ترى سجلات (بيانات ward ناقصة في جدول nurses).

الحل: ملء seed data بشكل كامل، تحسين SQL queries.

التأثير: بيانات موثوقة وسياسات دقيقة.

6. JWT Claims Accuracy

المشكلة: Pharmacist لا تملك permissions في JWT رغم وجودها في DB.

الحل: استعلام صحيح يجلب كل الصلاحيات من role_permissions.

التأثير: صلاحيات دقيقة في كل طلب.

7. Error Handling

المشكلة: رد غير واضح عند (UNIQUE violation (PostgreSQL error 23505).

الحل: معالج خاص لـ (SQL error codes (23505 duplicate, 23514 check violation).

التأثير: رسائل خطأ واضحة للمستخدم.

الدرس المستفاد

هذه السيناريوهات توضح أن النظام خضع لاختبار دقيق وحل مشاكل حقيقية بطريقة منهجية، مما يعكس ممارسات هندسية عالية الجودة.

12 الأعمال المستقبلية

المرحلة الأولى: تحسينات الأمان (Q3 2026)

- **Refresh Token Pattern:** accessToken 15 دقيقة، refreshToken أيام محفوظ في DB
- **JWT Blacklist:** جدول لإبطال tokens عند logout
- **HTTPS + HSTS:** بيئة الإنتاج فقط
- **Rate Limiting محسن:** Redis مع per-user tracking

المرحلة الثانية: ميزات وظيفية (Q4 2026)

- **نظام الإشعارات:** جدول notifications + WebSocket
- **تقرير شاملة:** appointments occupancy, billing revenue, audit summary
- **تقييد Nurse بالقسم:** assigned_wards array, shift_type

المرحلة الثالثة: توسع تكنولوجي (Q1 2027)

- **Docker + Kubernetes:** deployment scaled
- **Cloud Deployment:** AWS RDS + Heroku/GCP
- **Monitoring:** Datadog/New Relic + ELK Stack
- **CI/CD محسن:** Terraform, automated scaling

المرحلة الرابعة: التكامل (Q2-Q3 2027)

- **HL7/FHIR Integration:** تصدير سجلات بمعايير دولية
- **Payment Gateway:** Stripe/PayPal integration
- **Mobile App:** React Native/Flutter
- **Advanced Analytics:** لوحات تحليلات متقدمة

الامتثال القانوني

- **HIPAA:** للمنصات الأمريكية
- **GDPR:** للعملاء الأوروبيين
- **القوانين المحلية:** حماية البيانات بالعراق
- **النسخ الاحتياطية:** يومية مع اختبار استعادة
- **شهادات production:** SSL/TLS فقط

13 الخاتمة والإنجازات

ملخص الإنجازات

نظام إدارة مستشفى العالمية IVF الآمن يجمع بين:

1. الأمان المتعدد الطبقات

- RBAC: 7 أدوار، 68 صلاحية، least privilege
- RLS: 21 سياسة على 7 جداول حساسة
- JWT: توكنات 15 دقيقة مع permissionsg roles
- bcrypt: 12 جولة (0.3 ثانية/محاولة)
- Audit Logging: append-only مع old_values/new_values
- Data Masking: إخفاء أرقام حساسة حسب الدور
- Helmet + CORS + Rate Limiting: دفاع HTTP شامل

2. الموثوقية

- 61 اختبار ناجح مع GitHub Actions CI/CD
- **idempotent database scripts** آمنة للتشغيل المتكرر
- **معالجة أخطاء موحدة** مع تمييز بين AppError والخطأ البرمجي
- **Transaction safety** ضمان سلامة البيانات
- **معالجة NULL آمنة** في GUC و JSONB

3. الهندسة الدقيقة

حل 7 مشاكل واقعية بطريقة منهجية توضح ممارسات عالية الجودة.

النتائج الكمية

| المقياس | القيمة |

|-----|-----|

| الجداول | 22 |

| الأدوار | 7 |

| الصلاحيات | 68 |

| سياسات | 21 | RLS

| +Endpoints | 30 |

| الاختبارات | 61 | ✓

| Triggers | 22 |

الاستخدامات

1. نموذج تعليمي: RBAC, RLS, JWT, أمان التطبيقات

2. قاعدة للمشاريع: استخراج الأنماط والحلول

3. نموذج للمستشفيات: مع تكييفات محلية

4. عينة مهنية: توضيح قدرات هندسية

المراجع

• PostgreSQL RLS Documentation

• Express.js Security Best Practices

• OWASP Top 10

• JWT.io Standard

• bcryptjs Documentation

GitHub Repository: <https://github.com/ABOODE39/hospital-dbms-rbac>

CI/CD Status: GitHub Actions passing (61 اختبار)

آخر تحديث: 30 يونيو 2026